PRADITA University

IF140303-Web Application Development

Session-03:

Elixir Documentation and Testing

PRU/SPMI/FR-BM-18/0222

Alfa Yohannis



Contents

PRADITA University

1. Documentation in Elixir
   - Module-level Documentation
   - Function-level Documentation
2. Adding `ex_doc` for Documentation
   - Steps to Add `ex_doc`
3. Generating HTML Documentation
4. Unit Testing in Elixir
   - Unit Test in a Separate File
   - Function Documentation Tests (Doctest)
   - Running Tests from the Command Prompt
   - Running Tests for a Specific Function
5. Exercises
   - Writing Module Documentation
   - Writing Unit Tests for the Module
   - Running Tests for the Module
   - Running a Specific Test Function
6. Summary

Documentation in Elixir

PRADITA University

Elixir provides simple support for documentation in modules and functions.

- Documentation is written directly in the code using special annotations.
- Use `@moduledoc` to document a module.
- Use `@doc` to document functions.
- Documentation can be exported to HTML using the command:

```
1 mix docs
```

This command generates documentation in HTML format, which can be viewed in a browser.

Module-level Documentation

PRADITA University

To provide a description of the purpose, usage, and examples for a module, use the `@moduledoc` annotation.

```elixir
defmodule ExampleModule do
  @moduledoc """
  This module is an example of how to document a module in Elixir.

  It demonstrates how module documentation can be written
  and used as a reference when generating HTML documentation.
  """

  # Other functions and logic can be written here
end
```

Function-level Documentation

PRADITA University

In addition to modules, each function can also have documentation with the `@doc` annotation. This usually describes:

- The purpose of the function.
- The parameters required.
- The return value.
- Examples of usage in IEx.

```elixir
defmodule ExampleModule do
  @moduledoc """
  This module is an example of how to document a
      module in Elixir.
  """

  @doc """
  This function adds two numbers.

  ## Parameters
  - a: the first number.
  - b: the second number.

  ## Example
      iex> ExampleModule.add(2, 3)
      5
  """
  def add(a, b) do
    a + b
  end
end
```

Adding `ex_doc` (Part 1)

PRADITA University

To generate project documentation in Elixir, you need to add the `ex_doc` library.

1. Open the `mix.exs` file in your project, then add `ex_doc` to the list of dependencies in the `deps` function. Make sure it is included only in the `dev` environment.

```elixir
defp deps do
  [
    {:ex_doc, "~> 0.34", only: :dev, runtime: false}
  ]
end
```

2. Run the following command to download and install dependencies:

```
mix deps.get
```

Adding `ex_doc` (Part 2)

PRADITA University

3. After the dependency is installed, generate the documentation with the command:

```
mix docs
```

4. The generated documentation will be stored in the `doc/` folder. Open the `index.html` file in a browser to view the HTML documentation.

Note

`ex_doc` allows you to generate documentation in multiple formats (HTML, EPUB, PDF), making it very useful for both open-source and internal projects.

Generating HTML Documentation

PRADITA University

To generate documentation in HTML format, Elixir provides the simple command `mix docs`. This command collects documentation from modules and functions, and converts it into human-readable HTML pages.

1. Make sure you are in the Elixir project directory.
2. Run the following command in the terminal:

```
mix docs
```

3. The HTML documentation will be stored in the `doc/` folder. Open the `index.html` file in a browser.

Benefit:
HTML documentation makes it easier for others (or yourself in the future) to understand the code structure, functions, and how to use the modules.

Unit Testing in Elixir

PRADITA University

Unit testing in Elixir can be done using the built-in module `ExUnit`.

- Tests are written in dedicated files ending with `_test.exs`.
- Test modules use `ExUnit.Case` as a template.
- Supports `assert`, `refute`, and additional verification features.
- Function documentation can also be tested with the `doctest` annotation.

Benefit: Unit tests ensure that functions work as expected, and help prevent bugs when making code changes.

Unit Test in a Separate File

PRADITA University

Example of a test file inside the `test/` folder with the name `example_module_test.exs`:

```elixir
defmodule ExampleModuleTest do
  use ExUnit.Case

  test "adds two numbers" do
    assert ExampleModule.add(2, 3) == 5
  end

  test "subtracts two numbers" do
    assert ExampleModule.subtract(5, 2) == 3
  end
end
```

Running the Tests: Execute all tests with the command:

```
mix test
```

Unit Tests in Function Docs (Doctest)

Elixir also allows you to write unit tests directly in function documentation using the `doctest` annotation. These tests are executed automatically when `ExUnit` runs.

Advantages:

- Documentation and tests always stay in sync.
- Code examples in `@doc` are actually verified.
- Easy for readers to understand and confirm correctness.

Example Module with Doctest:

```elixir
defmodule ExampleModule do
  @moduledoc """
  Demo module with documentation and tests.
  """

  @doc """
  Adds two numbers.

  ## Example
      iex> ExampleModule.add(2, 3)
      5
  """
  def add(a, b), do: a + b

  @doc """
  Subtracts two numbers.

  ## Example
      iex> ExampleModule.subtract(5, 2)
      3
  """
  def subtract(a, b), do: a - b
end
```

Enabling and Running Doctest

PRADITA University

To enable `doctest`, add it inside your test module:

```elixir
defmodule ExampleModuleTest do
  use ExUnit.Case
  doctest ExampleModule
end
```

Execution: Run the following command in the terminal:

```
mix test
```

Doctest will execute the example code written in the documentation and verify that the results match the expected output.

Running Tests from the Command Prompt

Elixir makes it easy to run tests directly from the command prompt. There are several options depending on your needs:

Run All Tests

```
mix test
```

This command executes all tests inside the `test` folder and displays the results in the terminal.

Run Tests in a Specific File

```
mix test test/example_module_test.exs
```

This command executes tests only from a specific file, for example `ExampleModuleTest`.

Running Tests for a Specific Function

RADITA
niversity

You can run tests for a specific function in a module by using a **line number tag**.

Example: `add` function If the test for `add` is on line 4:

```
mix test test/
    example_module_test.exs:4
```

Example: `subtract` function If the test for `subtract` is on line 8:

```
mix test test/example_module_test.
    exs:8
```

With this approach, only the test on that line will be executed, making debugging and partial testing easier.

Exercise 1: Writing Module Documentation

Instructions:

- Create an Elixir module named `Calculator`.
- Add two functions: `multiply/2` and `divide/2`.
- Use the `@doc` annotation to write documentation for each function.
- Include usage examples (doctest) for each function.

Code Example:

```elixir
defmodule Calculator do
  @moduledoc """
  A module for basic arithmetic operations.
  """

  @doc """
  Multiplies two numbers.

  ## Example
      iex> Calculator.multiply(4, 5)
      20
  """
  def multiply(x, y), do: x * y

  @doc """
  Divides two numbers.

  ## Example
      iex> Calculator.divide(10, 2)
      5
  """
  def divide(x, y), do: x / y
end
```

Exercise 2: Writing Unit Tests for the Module

Instructions:

- Create a file named `calculator_test.exs` in the `test` folder.
- Use `ExUnit.Case` and `doctest`.
- Write tests for:
  - Multiplying two numbers.
  - Dividing two numbers.
  - Division by zero (should raise an error).

Example File:

```elixir
defmodule CalculatorTest do
  use ExUnit.Case
  doctest Calculator

  test "multiplies two numbers" do
    assert Calculator.multiply(4, 5) == 20
  end

  test "divides two numbers" do
    assert Calculator.divide(10, 2) == 5
  end

  test "divides by zero raises an error" do
    assert_raise ArithmeticError,
      fn -> Calculator.divide(10, 0) end
  end
end
```

Exercise 3: Running Tests for the Module

After writing `calculator_test.exs`, run the unit tests for the `Calculator` module with the following command:

```
mix test test/calculator_test.exs
```

Output: The terminal will show the test results, the number of successful tests, and details if any test fails.

Exercise 4: Running a Specific Test Function

Add a new test in `calculator_test.exs` for the `multiply/2` function, then run that test using its line number.

Example File:

```elixir
defmodule CalculatorTest do
  use ExUnit.Case
  doctest Calculator

  test "multiplies two numbers" do
    assert Calculator.multiply(4, 5) == 20
  end

  # New test added for specific function
  test "checks multiply with negative numbers" do
    assert Calculator.multiply(-4, 5) == -20
  end
end
```

Running the Test: If the new test was added at line 6, run the following command:

```
mix test test/
    calculator_test.exs:6
```

This command will only run the test on that line, making debugging or partial testing easier.

Summary

PRADITA University

- **Documentation in Elixir**
  - Use `@moduledoc` for modules
  - Use `@doc` for functions
  - Generate HTML docs with `ex_doc` and `mix docs`
- **Exercises**
  - Create a module with documentation
  - Write unit tests in a separate file
  - Run tests for modules & specific functions
- **Unit Testing with ExUnit**
  - Write tests in `_test.exs` files with `ExUnit.Case`
  - Use `assert`, `refute`, `assert_raise`
  - Use `doctest` to verify examples in documentation
- **Running Tests**
  - `mix test` to run all tests
  - `mix test test/file_name.exs` for a specific file
  - `mix test file.exs:line` for a specific function